

Str, tableau et algorithmie



Activité 1 : Une chaîne en or

On a reçu le message suivant : "povmsdkdsyxc ! fyec kfoj nomrspbo vo wocckqo. fyec odoc voc mrkwzsyxc no xcq."

1. Créer une variable message de type chaîne de caractères qui contiendra ce message.

2. Afficher le contenu de la variable message.

3. Tester la commande suivante `print(message[1])`. Que s'affiche-t-il ? Que remarquez-vous ?

4. Afficher le premier caractère de la chaîne.

5. Afficher les 8 premiers caractères.

6. Existe-t-il une autre méthode (vue dans un cours précédent) pour afficher ces caractères sans copier/coller ?

7. Afficher chaque lettre du message une à une. Aide : la fonction `len()` permet d'obtenir la taille d'une chaîne de caractères.

8. Afficher une lettre sur deux.

La fonction Python `ord()` permet de transformer une lettre en un entier. La fonction Python `chr()` permet à l'inverse de transformer un entier en une lettre.

9. Écrire une fonction qui prend une lettre en entrée et un entier, et qui renvoie une nouvelle lettre (la lettre donnée en entrée - la valeur de l'entier). Cette fonction ne devra pas effectuer de décalage lorsque la lettre entrée est un espace ou une ponctuation.

10. Sachant que le décalage utilisé pour écrire le message est 10 et en utilisant votre précédente fonction, afficher le message décodé.

11. Une erreur s'est glissée dans le message. En utilisant l'indice, corriger la lettre. La bonne lettre est un 's'. Que se passe-t-il ?

12. En récupérant les lettres une à une dans le message d'origine, les ajouter dans une deuxième chaîne de caractères (vide au départ), à l'exception de la dernière qui sera remplacée par un 's'.



Cours 1 : Chaîne de caractères

Une chaîne de caractères (`str`) est un type dit **immutable**, c'est-à-dire qu'on ne peut modifier (muter/changer) un des caractères. On peut accéder à chacune des lettres à l'aide de crochets `[]`, en indiquant à l'intérieur de ces derniers, la position du caractère concerné (le premier caractère se trouvant en position 0), mais on ne peut pas directement affecter un nouveau caractère à cette même position.

```
1     chaine = "abracabra"
2
3     print(chaine[1]) # -> 'b'
4     chaine[1] = 'z' # -> impossible car type str est immutable
```



Exercice 1 : Hack-sans

Écrire, en pseudo-code, une fonction permettant de supprimer toutes les lettres accentuées dans une chaîne de caractères.



Cours 2 : Tableau

Les types de base permettent de représenter une donnée spécifique. Cela est suffisant lorsqu'un programme manipule un petit nombre de données. Lorsqu'un programme nécessite un grand nombre de données, il est intéressant de les regrouper dans une structure. On parle alors de types construits.

Un **tableau** est une structure permettant de stocker des éléments de **même type**. Un tableau se déclare entre crochets `[]`. Pour accéder à un élément d'un tableau, on note : `nom_variable[indice]`. L'**indice** correspond à la position de l'élément dans le tableau.

Un tableau se manipule similairement à une chaîne de caractères ; on utilise les indices pour accéder à une cellule du tableau et on parcourt chaque élément à l'aide d'une boucle et de la fonction `len()`.



Exercice 2 : Tableau de bord

1. Écrire le code permettant de créer un tableau contenant les caractères (lettres) présents dans le mot "anticonstitutionnellement".

2. Écrire une fonction, prenant en paramètre un tableau, celle-ci doit le parcourir pour compter le nombre d'occurrences (apparitions) de la lettre 'e', la valeur du compteur sera renvoyée.



Cours 3 : Liste

En Python, il existe une structure appelée **liste** permettant de regrouper les données d'un même type. C'est cette structure qui implémente les tableaux en Python. Une liste se définit entre `[]` et utilise la virgule `,` comme séparateur.



Activité 2 : Play Liste

1. Créer une liste vide stockée dans une première variable nommée `liste1`. Puis afficher sa taille.

2. Dans une variable nommée `liste2`, créer une liste contenant trois éléments de type entier.

3. Parcourir `liste2` et doubler la valeur de chaque entier.

Tester le code suivant :

```
for indice in range(len(liste2)):
    liste1 = liste1 + [liste2[indice]]
print(liste1)
```

4. Que permet de faire l'opérateur `+` entre deux listes ?

5. Écrire une fonction fusion prenant en paramètres deux listes et qui renvoie une nouvelle liste alternant les éléments de chaque liste. Dans le cas où les deux listes données en entrée ne font pas la même taille, la fonction affiche un message d'erreur. Exemple :

```
melange([1, 3], [2, 4])
>> [1, 2, 3, 4]
melange([1, 3], [4])
>> "Les listes doivent faire la même taille"
```



Activité 3 : F-list-bustier

1. Écrire une fonction `extremum` qui prend en paramètres une liste d'entiers et un booléen. Si le booléen est à vrai, alors la fonction renverra la plus grande valeur de la liste. Si le booléen est à faux, dans ce cas, la fonction renverra la plus petite valeur de la liste.

2. Écrire une fonction `recherche` qui prend en paramètres une liste d'entiers et un nombre entier. Cette fonction renvoie la position de cet entier dans la liste ou -1 si l'entier n'est pas présent dans la liste.

3. Écrire une fonction `supprimer` qui prend en paramètres une liste d'entiers et une position (entier). Cette fonction renverra une nouvelle liste contenant les éléments de la première liste sans l'élément à la position donnée.

4. (Pour les plus rapides) Écrire une fonction qui prend en paramètres une liste d'entiers ainsi qu'un booléen et qui renvoie une nouvelle liste contenant les éléments de la première liste triée par ordre croissant si le booléen est à vrai, par ordre décroissant si le booléen est à faux. N'hésitez pas à utiliser les fonctions précédentes.



Activité 4 : List-bonne

L'office de tourisme de la ville de Lisbonne souhaite organiser une chasse au trésor dans la ville pour inciter les touristes à visiter des lieux touristiques moins fréquentés. Pour se faire, il souhaite cacher le nom de ces monuments dans un mot-croisé. Certains mots doivent être écrits à l'envers.

Écrire une fonction prenant en paramètres une liste de chaînes de caractères, et un nombre. La fonction devra remplacer autant de mots que donné en paramètre par ces mêmes mots à l'envers et afficher la liste.

Exemple:

```
miroir(["sintra", "pena", "cascais"], 2)
>>> ["artnis", "anep", "cascais"]
```



Activité 5 : Écris List-oire

Un professeur d'histoire souhaite calculer la moyenne des notes obtenues par ses élèves au dernier bac blanc. Pour cela, écrire une fonction prenant en paramètre une liste de notes (type float) et qui renverra la moyenne obtenue (float). Pour arrondir au centième près, vous pouvez utiliser la fonction round, dont un exemple d'utilisation vous est donné ci-dessous.

```
moyenne = 8.2356
moyenne = round(moyenne, 2)
print(moyenne)
>>> 8.23
```

Exemple de tableau de notes :

```
notes = [13.25, 14.50, 18.00, 20.00, 2.00, 5.75]
```



Activité 6 : Ba-list-ique

Sur une scène de crime, la police récupère des éclats de balle dans un mur. Pour essayer d'identifier l'arme, différentes armes à feu sont utilisées dans des conditions similaires, pour comparer les éclats. Le labo renvoie une liste indiquant pour chaque arme si les éclats correspondent.

Écrire une fonction prenant en paramètre une liste de booléen, et renvoyant un tableau contenant tous les indices des armes dont le test a été concluant (True).



Activité 7 : Black list

Mauricette souhaite organiser son anniversaire. Secrètement, elle a réussi à obtenir la liste de cadeaux de chaque invité ainsi que sa valeur. Cependant, Mauricette est une diva et ne souhaite pas recevoir de cadeau de valeur inférieur à 110€. Elle souhaite annuler l'invitation des invités qui ne la considère pas à sa juste valeur.

1. Créer deux listes de même taille, la première contenant les invités, la seconde contenant la valeur du cadeau de chaque invité.

2. Écrire une fonction qui prend en paramètres la liste des invités, la liste des prix. Cette fonction renvoie la liste des invités dont la valeur du cadeau est inférieure à 110€.

3. Créer une fonction qui compte le nombre de cadeaux de Mauricette va recevoir.



Activité 8 : To Do List

René a des problèmes de procrastination, et ne réussit à avancer dans aucune des tâches qu'il doit réaliser. Il décide donc de s'inspirer de la méthode Pomodoro pour se motiver, c'est-à-dire faire une liste de ses tâches avec le temps qu'il doit y consacrer, travailler 25 minutes sur la première tâche, la passer à la fin de la liste et travailler sur la tâche suivante, et ainsi de suite jusqu'à avoir tout terminé.

Écrire une fonction pomodoro prenant en paramètres un tableau de chaînes de caractères (une liste de tâches) et un tableau d'entiers (le temps en minutes de chaque tâche). Cette fonction devra afficher l'ordre dans lequel les tâches seront faites en suivant la méthode Pomodoro. Exemple :

```
pomodoro(["Jouer échecs", "Réviser NSI", "Aspirateur", "Vaisselle"], [45, 70, 20, 28])
>> "Jouer échecs", "Réviser NSI", "Aspirateur", "Vaisselle", "Jouer échecs", "Réviser NSI",
    "Vaisselle", "Réviser NSI"
```



Cours 4 : Parcours

Il est possible de parcourir une structure (liste, chaîne de caractères, etc.) de deux manières différentes en Python. La première sur laquelle nous avons travaillé précédemment à base de : `for ... in range(len(...))`. De cette façon, on parcourt notre structure à l'aide d'un indice permettant de savoir à quelle position on travaille et qui s'utilise entre crochets `[]`. Exemple :

```
notes = [12, 15, 19, 13]
for position in range(len(notes)):
    print(notes[position])
```

Il existe une seconde méthode permettant de travailler sur les valeurs d'une liste sans avoir besoin de connaître l'indice. Pour cela, on utilise simplement le mot-clé `in` : `for ... in ...`. De cette façon, on récupère les valeurs de notre structure sans avoir besoin de l'indice. On utilise, généralement, cette façon de faire quand on a besoin d'utiliser les valeurs sans modifier l'intérieur de notre structure (par exemple pour la calcul d'une moyenne). Exemple :

```
notes = [12, 15, 19, 13]
for note_eleve in notes:
    print(note_eleve)
```

Même si la deuxième façon est moins "sophistiquée" que la première, ne pas connaître la position des éléments parcourus pourra s'avérer problématique dans certaines fonctions. Attention donc à choisir une méthode de parcours adaptée à ce que vous souhaitez faire dans le reste du programme.

Enfin, le mot-clé `in` a une autre utilité en Python. Il sert à regarder l'appartenance d'une valeur dans une structure,

c'est-à-dire vérifier si une valeur est présente ou absente. On peut donc utiliser ce mot-clef dans des structures conditionnelles. Exemple :

```
13 in [12, 15, 19, 13]
>> True

notes = [12, 15, 19, 13]
if (20 in notes):
    print("Bravo")
else:
    print("Une prochaine fois")
```



Exercice 3 : Parcours du combattant

Dans chacune des situations, indiquer quel type de parcours est à privilégier et pour quelle raison, puis écrire le code du parcours.

1. Récupérer un maximum dans une liste.

2. Connaître toutes les positions d'un nombre dans une liste.

3. Multiplier les valeurs d'une liste par un entier n (exemple n = 2).

4. Échanger la position de la plus grande valeur et de la plus petite valeur d'une liste.

5. Faire la somme des éléments d'une liste.



Cours 5 : Liste en compréhension

En Python, il est possible de générer directement une liste à partir d'une itération.

```
# exemple :
liste_comp = [i for i in range(5)]

# replace
```



```
ma_liste = []
for i in range(5):
    ma_liste = ma_liste + [i]
```

Cette méthode d'écriture permet notamment de générer une nouvelle liste à partir d'une liste déjà existante.

```
ma_liste = [0, 1, 2, 3]

liste_des_carres = [nb * nb for nb in ma_liste]
```

Enfin, il est possible d'appliquer des conditionnelles, ou d'autres itérateurs sur le liste générée.

```
ma_liste = [0, 1, 2, 3]

liste_pairs = [nb for nb in ma_liste if nb % 2 == 0]
```



Activité 9 : Je vous ai compris

1. Créer, grâce à une liste en compréhension, une liste contenant les 20 premiers nombres non nuls.

2. On considère une liste de booléen. Créer, grâce à une liste en compréhension, une liste contenant tous les indices des éléments valant True dans la liste.

3. On considère deux listes de même taille. La première liste contient des booléens, et la deuxième des entiers. Créer, grâce à une liste en compréhension, une liste contenant tous les éléments de la deuxième liste si les éléments aux mêmes indices dans la première valent True. Exemple:

```
# pour les listes suivantes
l1 = [True, False, False, True, False]
l2 = [1, 2, 3, 4, 5]

# on veut la liste
res = [1, 4]
```



Activité 10 : Spécia-Liste

1. Créer une fonction qui prend en paramètres une liste et un nombre. Cette fonction renverra la liste avec le nombre ajouté en dernière position. Exemple :

```
ajouter_fin([1, 2, 7], 5)
>> [1, 2, 7, 5]
```

2. Écrire une fonction prenant en paramètre une liste d'entiers, une position et un entier. Cette fonction renvoie une nouvelle liste en y ajoutant l'entier passé en paramètre à la position donnée en paramètre. Exemple :

```
ajouter_position([1, 2, 7], 0, 5)
>> [5, 1, 2, 7]
```

3. Écrire une fonction prenant en paramètre une liste d'entiers. Cette fonction renvoie une nouvelle liste sans l'élément se trouvant à la fin de la liste. Exemple :

```
supprimer_dernier([1, 2, 7, 5])
>> [1, 2, 7]
```

4. Écrire une fonction prenant en paramètre une liste d'entiers et une position. Cette fonction renvoie une nouvelle liste sans l'élément se trouvant à la position donnée en paramètre. Exemple :

```
supprimer_position([1, 2, 7, 5], 2)
>> [1, 2, 5]
```

5. Écrire une fonction prenant en paramètre une liste d'entiers et un entier. Cette fonction renvoie la liste en supprimant toutes les valeurs égales à l'entier passé en paramètre. Exemple :

```
supprimer_position([1, 2, 7, 2], 2)
>> [1, 7]
```



Cours 6 : Méthodes sur les listes

Comme nous l'avons vu, une liste est une structure mutable (modifiable), plusieurs opérations sont donc possibles sur ces dernières. Il existe pour cela, ce que l'on appelle des méthodes (des actions possibles prédéfinies), nous allons les détailler ci-dessous.

- `append()` : permet d'ajouter un élément à la fin de la liste.

```
legumes = ["carotte", "courgette"]
legumes.append("asperge")
print(legumes)
```

```
>>> ["carotte", "courgette", "asperge"]
```

- `insert(indice, élément)` : permet d'insérer un élément à une position donnée dans une liste.

```
legumes = ["carotte", "courgette"]
legumes.insert(1, "asperge")
print(legumes)
>>> ["carotte", "asperge", "courgette"]
```

- `remove(élément)` : permet de supprimer la première occurrence d'un élément.

```
legumes = ["carotte", "courgette"]
legumes.remove("carotte")
print(legumes)
>>> ["courgette"]
```

- `pop(indice)` : permet de supprimer un élément de la liste et le renvoyer, on peut préciser l'indice de l'élément mais par défaut cela sera le dernier.

```
legumes = ["carotte", "courgette", "asperge"]
legumes.pop(1)
print(legumes)
>>> ["carotte", "asperge"]
```

- `index(élément)` : permet de donner la position de la première occurrence d'un élément dans une liste.

```
nombres = [4,6,3,8,9,3]
print(nombres.index(3))
>>> 2
```

- `len(liste)` : permet de donner la taille d'une liste.

```
nombres = [4,6,3,8,9,3]
print(len(nombres))
>>> 6
```



Activité 11 : Méthod-ique

1. Écrire une fonction prenant en paramètres une liste et une valeur, qui déplacera la première occurrence de cette valeur à la fin de la liste.

2. Écrire une fonction qui renverra la fusion (une liste) de deux listes d'entiers **triées** (dans l'ordre croissant) passées en paramètres.

Objectifs :

Vous êtes désormais capables :

- de parcourir des chaînes de caractères ou des listes ;
- de créer des listes par compréhension ou par extension ;
- d'expliquer ce qu'est une structure immutable et de citer un exemple ;
- de modifier les éléments d'une liste ;
- de donner la taille d'une liste ou d'une chaîne de caractères ;
- de vérifier l'appartenance d'un élément à une liste ou une chaîne de caractères ;
- d'utiliser les méthodes disponibles sur les listes.